

PROJECT REVIEW REPORT

On

GRAPH-BASED REAL-TIME UPI FRAUD DETECTION AND SCAM LINK/QR IDENTIFIER SYSTEM USING AI AND ML

Submitted for Partial Fulfillment of Award of

BACHELOR OF TECHNOLOGY

In

Information Technology

Submitted by

Name of the Student	Register No.
<Name of the student>	<Register No.>
<Name of the student>	<Register No.>
<Name of the student>	<Register No.>
<Name of the student>	<Register No.>

Under the Guidance of

<Project Supervisor>

<Designation of the project supervisor>

Department of Information Technology

Sagi Rama Krishnam Raju Engineering College (A)

SRKR Marg, Bhimavaram, Pin: 534204, West Godavari, Andhra Pradesh, India

Affiliated to

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, KAKINADA

<Academic Year 2026 - 2027>

RECOMMENDATION

I hereby recommend that the project titled “Graph-Based Real-Time UPI Fraud Detection and Scam Link/QR Identifier System Using AI and ML” submitted by <Name of project group members> be accepted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology, Sagi Rama Krishnam Raju Engineering College (A), Bhimavaram, affiliated to Jawaharlal Nehru Technological University, Kakinada.

The work reported here was carried out by the candidates under my supervision. The system described in this report has been implemented, tested and evaluated by them, and the results reported in Chapter 8 are reproducible from the artefacts submitted with it.

Place:

Date:

Project Supervisor

ABSTRACT

The Unified Payments Interface settled 24.51 billion transactions worth Rs 29.82 trillion in August 2026, an average of 791 million payments a day. A UPI payment is irrevocable: once the money moves there is no chargeback, so a fraud system that classifies stored transactions produces an accurate description of a loss that has already happened. The dominant loss pattern in India is also the one such a system is least able to see - the account holder authorises the payment themselves, having been persuaded to, so nothing about the payer's behaviour is anomalous.

This project builds a privacy-aware, pre-payment risk intelligence system for UPI. Before any money moves, a scanned QR code, a pasted UPI ID or a phone number is assessed by ten independent evidence families: the structure of the address and the payload it arrived in, the payee's aggregate reputation, the payer's own behavioural baseline, the relationship between the two, the amount in absolute terms, the purpose the payer states, coercive framing in any message that prompted the payment, the reputation of links carried alongside it, a calibrated behavioural classifier, and the payee's position in an incrementally maintained payer-payee graph. Each family returns a score, an availability flag and a set of typed facts. Scores are merged by a damped-maximum rule, and corroboration between families is rewarded only when they rest on genuinely distinct observations, so two modules reading the same timestamp cannot be counted as two agreeing witnesses.

The system's central design commitment is that absence of evidence is never evidence of safety. Unknown, observed, registered and provider-verified are four distinct states, and a family that cannot measure something reports itself unavailable rather than contributing a zero. Every assessment therefore carries two numbers - a risk score and a confidence score derived from evidence coverage - and one verdict vocabulary: APPROVE, WARN, STEP_UP or BLOCK, produced by a single mapping.

The behavioural classifier is a histogram gradient-boosted ensemble with isotonic calibration, trained on a 120,000-row simulator in which labels come from the generative process rather than from a rule over the features, and in which the payee's history is deliberately unobservable on 24.5% of rows because that is the condition the live system operates under. Measured on a held-out test split with the operating threshold chosen on validation, it reaches a PR-AUC of 0.486 and 49.3% recall at a 0.95% realised false-positive rate; recall is 94.5% on account takeover and 19.2% on victim-authorized social engineering, and using both halves of the feature set raises social-engineering recall 7.7 percentage points above the better half alone. The implementation is a React and TypeScript client, an Express authentication and proxy layer, a FastAPI scoring service and a single SQLite store, covered by 581 automated tests, and it is delivered as a web application and a Capacitor-packaged Android build.

The evaluation uses simulated payment data; no public labelled UPI fraud corpus exists, and the system holds no NPCI, payment-service-provider or banking integration. Every figure in this report describes the method on that data and on a reproducible demonstration statement, not field performance.

Keywords: UPI, pre-payment risk assessment, authorised push payment fraud, evidence availability, calibrated classification, graph-based fraud detection, QR and scam-link analysis, explainable risk scoring.

TABLE OF CONTENTS

DESCRIPTION	PAGE NUMBER
RECOMMENDATION BY SUPERVISOR.....	i
ABSTRACT.....	ii
1. Introduction.....	1
2. Literature Survey.....	4
3. Research Gaps.....	8
4. Objectives.....	10
5. Hardware / Software Requirement with Specifications.....	12
6. Methodology.....	15
7. Architecture (Implementation and Testing).....	23
8. Result and Analysis.....	28
9. Conclusion and Future Scope.....	33
10. References.....	36

LIST OF TABLES

Table 2.1 Literature survey.....	4
Table 5.1 Software stack and versions.....	12
Table 6.1 Evidence families, weights and facts.....	15
Table 6.2 Feature set and availability.....	15
Table 7.1 Database schema.....	23
Table 7.2 Test suite composition.....	23
Table 8.1 Classifier performance.....	28
Table 8.2 Recall by fraud scenario.....	28
Table 8.3 Feature-half ablation.....	28
Table 8.4 End-to-end demonstration results.....	28

LIST OF FIGURES

Figure 6.1 Evidence assembly and the verdict pipeline.....	15
Figure 7.1 System architecture.....	23
Figure 7.2 Pre-payment check sequence.....	23

1. INTRODUCTION

1.1 Background

The Unified Payments Interface (UPI), built by the National Payments Corporation of India (NPCI), has become the default way money moves between people and small businesses in India. In August 2026 it carried 24.51 billion transactions worth Rs 29.82 trillion, a daily average of 791 million payments, growing 22% in volume year on year. Since 15 September 2025 a person-to-person transfer may carry up to Rs 1,00,000, while payments to verified merchants in specified categories may carry up to Rs 5,00,000 per transaction, with daily ceilings as high as Rs 10,00,000 in categories such as insurance and capital markets.

Two properties of the rail define the problem this project addresses. The first is irrevocability. A UPI payment is a push transfer that settles in seconds and has no chargeback mechanism comparable to a card scheme's; once a payer has approved it, recovery depends on the recipient's bank freezing an account that, in the fraud cases that matter, has usually already been emptied. The second is that the payer is frequently a willing participant. In the dominant Indian fraud pattern the account holder is persuaded - by a phone call claiming to be from a bank, a fake customer-care number, a refund that requires a small fee first, a QR code presented as a way to receive money - and then authorises the payment themselves, with their own PIN, on their own device, at an ordinary hour.

These two facts together determine where a fraud system must sit. A classifier that scores transactions after they are recorded can describe losses but cannot prevent them, and a classifier that models only the payer's own behaviour is structurally blind to a payment the payer made deliberately. What is left is the counterparty and the payment's context: who is being paid, what is known about that address, how it sits in the wider payment graph, what arrived alongside the request, and whether the payer's stated purpose is consistent with the address receiving the money.

1.2 Motivation

The motivation for this work came from three observations that recur in the reported experience of UPI users and in the public record. First, the moment at which a user could still be helped is the moment before they press the pay button, and at that moment the only information available is about the payee and the payload - not about an outcome. Second, the systems that can see the payee's history in aggregate are the ones users cannot query: a payment service provider knows that an address is four days old and has taken money from nine people who never paid it again, and the payer knows nothing. Third, and most importantly for the engineering, a risk system built on partial information is dangerous in a specific and quiet way. If an address about which nothing is known is scored the same as an address with two years of good history, then the defining property of a collection account - that it has no history - is exactly what makes it look safe.

That last observation shaped the entire project. It is easy to build a system that outputs a single risk percentage; it is considerably harder, and far more useful, to build one that distinguishes “we looked and found nothing wrong” from “we could not look”, and that says which of the two it is doing on every field of every response.

1.3 Problem Statement

Design and implement a system that, given a scanned QR code, a pasted UPI ID or a phone number, together with an optional amount, purpose and message, produces - before any money moves and within a few hundred milliseconds - a risk assessment that:

- draws on multiple independent evidence families rather than a single model, so that a payment can be stopped for a reason a reader can inspect;
- never treats missing information as favourable information, and reports separately how bad the available evidence looks and how much evidence there was;
- distinguishes an unknown entity from an observed one, a registered one and one verified by an external provider, and never claims a verification it did not perform;
- uses only information that exists before the decision, both in the live path and in the training data;
- issues one verdict in one vocabulary, produced in one place, so that no screen or endpoint can disagree about what a score means.

1.4 Scope of the Project

The system is a decision-support application for an individual payer. It does not initiate, authorise or settle payments, and it is not a bank or a payment service provider. Its knowledge of payees is built from two sources: statements that its own users import, and payments its own users confirm. It therefore knows nothing about an address that none of its users has ever paid, and the system is designed to say so rather than to guess. The project holds no NPCI, PSP or banking integration, and every claim in this report is scoped to what the implementation can actually observe.

1.5 Contributions

1. An evidence-availability model in which NaN means “not measurable” and zero means “measured as none”, carried end to end from the dataset generator through the feature vector, the classifier, the evidence rows and the API contract, and enforced at construction time so that an unavailable family cannot carry a score.
2. A fact-aware corroboration rule: families declare the observations their scores rest on, and the agreement bonus counts distinct facts bounded by the number of reporting families, which removes the double counting that two modules reading one timestamp would otherwise earn.

3. An incremental graph intelligence layer - a persisted union-find structure and a per-payee cache updated when a payment is observed - that gives a bounded two-hop payer-overlap reading on the real-time path without rebuilding a graph per request.
4. A simulator whose labels come from four distinct fraud behaviours rather than a threshold rule, in which legitimate and fraudulent payments overlap deliberately, and in which the payee's history is unobservable on a realistic fraction of rows.
5. An honest evaluation discipline: PR-AUC and recall at a fixed false-positive budget rather than accuracy, the operating threshold chosen on validation and measured once on test, a payer-half versus payee-half ablation, and a published availability-bias figure that bounds how much “unknown” is acting as a proxy for “fraud”.

1.6 Organisation of the Report

Chapter 2 surveys the literature on payment-fraud detection, graph-based methods, authorised-push-payment scams, calibration and interpretability, and synthetic payment data. Chapter 3 states the gaps this project addresses. Chapter 4 sets the objectives. Chapter 5 lists hardware and software requirements. Chapter 6 develops the methodology - the entity-status model, the evidence families, the combination rule, the amount model, the graph layer and the machine-learning pipeline. Chapter 7 describes the architecture, the implementation and the testing strategy. Chapter 8 reports results and analyses them, including the limitations the measurements expose. Chapter 9 concludes and sets out future scope, and Chapter 10 lists the references.

2. LITERATURE SURVEY

The work relevant to this project falls into five groups: supervised detection on payment transaction tables, graph-based fraud detection, the authorised-push-payment literature, the methodology of evaluating and calibrating rare-event classifiers, and synthetic payment data. Each group is summarised below and the individual works are tabulated in Table 2.1.

2.1 Supervised Detection on Payment Transaction Tables

The largest body of directly comparable work applies tree ensembles to tabular transaction features. Recent Indian journal literature on UPI specifically converges on decision trees, random forests, gradient boosting and stacked ensembles, and reports high accuracy on datasets that are either balanced by construction or synthesised without deliberate overlap between classes. LightGBM formalised the histogram-based gradient boosting that makes these models cheap to fit on wide tabular data, and also introduced the native handling of absent values by learned default split directions, which this project reuses for a different purpose than efficiency.

The methodological weakness across this group is consistent: features are computed from a completed transaction record, which means the classifier answers “was this payment fraudulent?” on evidence that includes the payment itself, and evaluation is reported as accuracy at a base rate where a constant “legitimate” prediction scores above 98%.

2.2 Graph-Based Fraud Detection

Fraud is relational, and the graph literature exploits that. FRAUDAR finds dense blocks in a bipartite graph with bounds that hold even when fraudsters add legitimate edges as camouflage. The Elliptic study showed that graph structure adds signal over local features for illicit-node classification while noting that gradient boosting remains a strong baseline. CARE-GNN addresses camouflage directly by learning which neighbours to aggregate.

All three are whole-graph methods evaluated in batch. Their cost scales with the graph, and none is formulated for the question this project must answer in a few hundred milliseconds: what is this one address's position in the graph, right now, given that the graph changes whenever a user confirms a payment. That gap motivated the incremental design in Section 6.7.

2.3 Authorised Push Payment Scams

The APP literature is the closest match to the Indian loss pattern. The Kansas City Fed briefing sets out why authorised scams resist behavioural detection - the payer is behaving normally by definition - and identifies counterparty verification and deliberate payment friction as the mechanisms that work. The Journal of Financial Regulation analysis compares liability regimes and finds that reallocating loss changes institutional incentives without warning the payer at the moment of payment. Together they establish the problem's shape but leave the payer-side computation unspecified, which is precisely where this project sits.

2.4 Evaluation, Calibration and Interpretability

Davis and Goadrich showed formally that dominance in ROC space does not imply dominance in precision-recall space under class skew, and Saito and Rehmsmeier demonstrated empirically that at low base rates the ROC view is optimistic while the precision-recall view tracks operational cost. Niculescu-Mizil and Caruana established that boosted trees are systematically over-confident and that isotonic regression corrects them when enough calibration data is available. Lundberg and Lee's SHAP provides per-prediction attributions, with the caveat - which cost this project a defect of its own - that attributions are meaningful only against a representative background distribution.

2.5 Synthetic Payment Data

PaySim remains the reference approach for the absence of public labelled payment data: simulate agents, inject fraud as agent behaviour, and derive labels from that behaviour rather than from a rule over the recorded features. This project follows the same principle and extends it in one respect that the literature does not address - the simulator also models what the deployment cannot see, hiding the payee's history on the fraction of rows where a real deployment would have no record of the address.

Table 2.1 Literature survey

S.No	Title	Author	Journal Name & Year	Methodology Adapted	Key Findings	Gaps
1	PaySim: A Financial Mobile Money Simulator for Fraud Detection	E. A. Lopez-Rojas, A. Elmir, S. Axelsson	Proc. 28th European Modeling and Simulation Symposium (EMSS), 2016	Agent-based simulation of mobile-money transfers; fraud injected as agent behaviour; classifiers trained on the synthetic log.	Establishes that a simulator with behaviour-derived labels is a legitimate substitute where no public labelled payment corpus exists.	Models an operator's own ledger with full visibility; no notion of a feature being unobservable at decision time.
2	Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics	M. Weber, G. Domeniconi, J. Chen, et al.	KDD Workshop on Anomaly Detection in Finance, 2019	Elliptic transaction graph; logistic regression, random forest and GCN compared on illicit-node classification.	Graph structure adds measurable signal over local features; gradient boosting remains a strong baseline against GNNs.	Retrospective, batch analysis of a settled ledger; no real-time pre-transaction path and no calibrated probabilities.
3	FRAUDAR: Bounding Graph Fraud in the Face of Camouflage	B. Hooi, H. A. Song, A. Beutel, et al.	Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, 2016	Dense-subgraph detection on a bipartite graph with provable bounds under adversarial camouflage.	Suspiciously dense payer-payee blocks can be found with guarantees even when fraudsters add legitimate-looking edges.	Whole-graph optimisation per run; cost grows with the graph and does not fit a per-payment latency budget.
4	Enhancing Graph Neural Network-based Fraud Detectors against Camouflaged Fraudsters (CARE-GNN)	Y. Dou, Z. Liu, L. Sun, et al.	Proc. 29th ACM Int. Conf. on Information and Knowledge Management (CIKM), 2020	Label-aware neighbour sampling and reinforcement-learned neighbour selection on relation graphs.	Camouflage degrades naive neighbour aggregation; selective aggregation restores performance.	Requires a trained graph model and full neighbourhoods at inference; assumes node features are always present.

S.No	Title	Author	Journal Name & Year	Methodology Adapted	Key Findings	Gaps
5	Isolation Forest	F. T. Liu, K. M. Ting, Z.-H. Zhou	Proc. 8th IEEE Int. Conf. on Data Mining (ICDM), 2008	Unsupervised anomaly detection by random isolation of points in randomly partitioned subspaces.	Anomalies are isolated in fewer splits; linear time and low memory make it viable as a second opinion on normal traffic.	Anomalous is not fraudulent; an unsupervised score cannot be calibrated to a probability of fraud on its own.
6	LightGBM: A Highly Efficient Gradient Boosting Decision Tree	G. Ke, Q. Meng, T. Finley, et al.	Advances in Neural Information Processing Systems (NeurIPS) 30, 2017	Histogram-based gradient boosting with gradient-based one-side sampling and exclusive feature bundling.	Histogram GBDT gives state-of-the-art tabular accuracy at low cost and handles missing values natively by learned default directions.	Native missing-value handling is presented as an efficiency device, not as a way to represent evidence that could not be measured.
7	Predicting Good Probabilities with Supervised Learning	A. Niculescu-Mizil, R. Caruana	Proc. 22nd Int. Conf. on Machine Learning (ICML), 2005	Comparison of Platt scaling and isotonic regression across classifier families; reliability diagrams.	Boosted trees are systematically over-confident; isotonic regression corrects them well when calibration data is sufficient.	Calibration is evaluated in isolation; it says nothing about what a probability means when some inputs are absent.
8	The Relationship Between Precision-Recall and ROC Curves	J. Davis, M. Goadrich	Proc. 23rd Int. Conf. on Machine Learning (ICML), 2006	Formal relation between the two curve families under class skew.	A dominant ROC curve need not dominate in precision-recall; PR is the informative view under heavy imbalance.	Methodological; applied fraud papers still frequently report accuracy or ROC-AUC alone.
9	The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets	T. Saito, M. Rehmsmeier	PLOS ONE, vol. 10, no. 3, 2015	Empirical study of evaluation metrics under extreme class imbalance.	ROC-AUC is optimistic at low base rates; PR-AUC and recall at a fixed false-positive budget reflect operational cost.	Does not address per-scenario reporting, which is where an averaged fraud metric hides the hard class.
10	A Unified Approach to Interpreting Model Predictions (SHAP)	S. M. Lundberg, S.-I. Lee	Advances in Neural Information Processing Systems (NeurIPS) 30, 2017	Additive feature attribution with Shapley values; a background distribution defines the reference.	Gives per-prediction attributions that are consistent and locally accurate, suitable for explaining an individual decision.	Attribution quality depends entirely on a representative background; a noise background yields meaningless explanations.
11	Combating Authorized Push Payment Scams in Fast Payment Systems	Y. Z. Toh	Payments System Research Briefing, Federal Reserve Bank of Kansas City, 2024	Policy and mechanism review of APP scams across fast-payment rails, including confirmation-of-payee and payment friction.	In authorised scams the payer's own behaviour is normal by construction; counterparty signals and pre-payment friction are the effective levers.	Discusses institutional and regulatory mechanisms rather than an implementable payer-side risk computation.
12	'Authorized Push Payment' Bank Fraud:	Oxford Academic	Journal of Financial Regulation, vol. 10,	Comparative regulatory analysis of	Liability frameworks shift incentives but do	Leaves the detection problem open: what a

S.No	Title	Author	Journal Name & Year	Methodology Adapted	Key Findings	Gaps
	What Does an Effective Regulatory Response Look Like?	(author per journal record)	no. 2, 2024	liability allocation for authorised transfers.	not by themselves warn the payer at the moment of payment.	system may claim about a counterparty without provider-level data.
13	Detection of Mule Accounts and Fraudsters in UPI Transactions Using AI and Machine Learning Techniques	Authors per journal record	International Research Journal of Engineering (IRE Journals), 2024	Supervised classification over transaction attributes to flag mule accounts in UPI-style logs.	Mule accounts exhibit characteristic inflow shapes - many one-time payers, short lifespan, little outflow diversity.	Post-hoc account-level classification; features assume a ledger-wide view unavailable to a payer's device.
14	UPI Fraud Detection Using Machine Learning / Enhancing UPI Fraud Detection Using Stacked Generalization	Multiple authors (IJIRT, IJRAR, IJMSM)	Indian journals, 2024-2025	Decision trees, random forests, XGBoost and stacked ensembles on transaction tables; accuracy-centred evaluation.	Tree ensembles perform well on tabular payment features; class imbalance handling materially changes reported results.	Accuracy on separable or balanced data; no pre-payment framing, no availability semantics, no calibration or FPR budget.
15	Unicode Security Mechanisms (Unicode Technical Standard #39)	Unicode Consortium	Unicode Technical Standard #39, current revision	Confusable-character mapping and restriction levels for identifiers.	Visually identical identifiers can be constructed across scripts; skeleton mapping detects them where normalisation does not.	Aimed at identifiers generally; payment-address handles and display names need a domain-specific application.

Note: Table 2.1 is reproduced in the project presentation as required by the departmental format.

3. RESEARCH GAPS

The survey in Chapter 2 identifies eight gaps. Each is stated as the gap, the consequence, and the response taken in this project.

3.1 Detection Happens After Settlement

Almost all reviewed work scores a transaction record. On an irrevocable rail this yields an accurate account of a completed loss. Response: the primary path is a pre-payment check on an address, a payload and an intended amount; no feature anywhere in that path reads the transaction being decided or anything later than it.

3.2 Payer-Only Modelling Cannot See Authorised Fraud

Behavioural models describe deviation from a payer's own norm, and in a social-engineering payment there is no deviation to describe. Response: the payer's behaviour is one evidence family among ten, and the ablation in Section 8.4 measures what the payer half can and cannot do rather than asserting it.

3.3 Missing Information Is Substituted With Favourable Defaults

Where a feature is unavailable, the common remedy is imputation - a mean, a median, or a hand-chosen constant. For payee history this is not a statistical inconvenience but an inversion: an address with no history is scored as though it had a moderate one, and having no history is the defining property of a collection account. Response: unobservable payee features are NaN in training and in serving, a companion availability feature states which case holds, and the API publishes null rather than a number. Section 6.4 develops this.

3.4 One Number Conflates Risk With Certainty

A single risk percentage cannot distinguish a well-supported all-clear from an assessment that could not be made. Response: every assessment carries a risk score and a confidence score computed from evidence coverage, plus an evidence level of FULL, PARTIAL or MINIMAL, and the families that could not be read are enumerated in the response.

3.5 Correlated Evidence Is Counted as Independent Corroboration

Ensembles and rule stacks reward agreement between components, but components frequently read the same underlying observation - two modules applying the same night-hour predicate to the same timestamp are one observation, not two. Response: families declare typed facts and the corroboration bonus counts distinct facts bounded by the number of reporting families (Section 6.6).

3.6 Graph Analytics Are Batch, Not Real-Time

Dense-block and GNN methods require the graph, which grows with use. Response: a persisted union-find structure and a per-payee cache are updated when a payment is observed, and the read path answers a bounded two-hop overlap query with no writes and no rebuild (Section 6.7).

3.7 Claims Exceed the Data Available to the System

Payer-side proposals in the applied literature frequently assume access to rail-level or bank-level data. Response: the system states an entity's status as UNKNOWN, OBSERVED, REGISTERED or VERIFIED, where VERIFIED requires an actual provider confirmation, and it claims nothing it did not observe. No NPCI, PSP or banking integration is claimed anywhere in this report.

3.8 Evaluation Is Not Operationally Meaningful

Accuracy at a 1% base rate, thresholds tuned on the test set, and averaged recall that hides the hardest class are all common. Response: PR-AUC and recall at a fixed 1% false-positive budget, the threshold chosen on a validation split and measured once on test, per-scenario recall reported separately, calibration reported as a Brier skill score, and a published availability-bias figure that bounds how far “unknown” could be acting as a proxy for “fraud”.

4. OBJECTIVES

4.1 Primary Objective

To design, implement and honestly evaluate a privacy-aware, real-time pre-payment risk intelligence system for UPI that assesses a payee address and its payload before money moves, combines multiple independent evidence families into one verdict, and never converts missing evidence into favourable evidence.

4.2 Specific Objectives

6. Model entity knowledge explicitly: represent a payee as UNKNOWN, OBSERVED, REGISTERED or VERIFIED, with VERIFIED reachable only through an external provider confirmation, and expose the resulting evidence level as FULL, PARTIAL or MINIMAL.
7. Implement ten independent evidence families - address and QR integrity, payee reputation, payer behaviour, payer-payee relationship, amount context, stated purpose, message pressure, link safety, a calibrated behavioural classifier and graph position - each returning a score, an availability flag and typed facts.
8. Combine families with a damped-maximum rule and a fact-aware corroboration bonus, so that a payment is stopped by agreement between genuinely independent observations rather than by one observation counted several times.
9. Preserve availability semantics end to end: NaN for unmeasurable, zero for measured-as-none, null on the wire, and a construction-time guarantee that an unavailable family carries no score.
10. Produce exactly one verdict vocabulary - APPROVE, WARN, STEP_UP, BLOCK - from exactly one score-to-band mapping, with any retained legacy label derived from it so the two cannot disagree.
11. Train a calibrated classifier on a simulator whose labels derive from four distinct fraud behaviours, whose classes overlap deliberately, and whose payee history is unobservable on a realistic fraction of rows; report PR-AUC, recall at a 1% false-positive budget, per-scenario recall and calibration.
12. Maintain graph intelligence incrementally so that a payee's component size and payer-overlap reading are available on the real-time path within the latency budget and without writes on the read path.
13. Detect scam links and QR-borne addresses, including Unicode-confusable and look-alike bank handles, and extract addresses from an uploaded or camera-captured QR image.
14. Deliver the system as a web application and a Capacitor-packaged Android build, with an explain-only assistant that can describe a verdict but cannot influence one.

15. Establish an automated test suite that pins the safety properties - availability, single verdict source, no post-decision information, consistent error responses - rather than only the happy paths.

4.3 Non-Objectives

- The system does not initiate, authorise, hold or settle payments.
- It does not claim, and must not be presented as having, NPCI, PSP or banking data access.
- It does not attempt to identify individuals; payer identifiers in aggregate views are opaque account identifiers.
- It does not replace the user's judgement: STEP_UP and BLOCK are advice to verify or to stop, delivered before a payment the user is still free to make.

5. HARDWARE / SOFTWARE REQUIREMENT WITH SPECIFICATIONS

5.1 Development Hardware

Item	Specification used	Minimum workable
Processor	x86-64, 4 cores, 2.4 GHz	Dual core, 2.0 GHz
Memory	16 GB	8 GB (model training peaks near 2 GB)
Storage	20 GB free (SSD)	10 GB free
Operating system	Windows 11 (64-bit); also verified on Linux	Windows 10 / Ubuntu 22.04
Android test device	Android 11+ with a working rear camera	Android 8.1 (Capacitor 8 floor)
Network	Broadband; required for package installation and deployment	Any internet connection

Training the classifier on 120,000 rows takes approximately two to three minutes on the development machine. The model is not committed to version control, because a scikit-learn pickle is tied to the NumPy and scikit-learn versions that produced it; it is regenerated with a single command after installation, and the exact versions used are recorded inside models/metrics.json.

5.2 Deployment Environment

- Client bundle: static hosting (Vercel free tier) serving the Vite production build.
- Scoring service: a single container (Render free tier) running Uvicorn with the FastAPI application; the free instance sleeps when idle, so the first request after a pause is slow.
- Store: one SQLite database file on the service's disk; the schema is created on first connection.
- Android: a Capacitor 8 wrapper around the same client bundle, with an offline evaluation path for use without the scoring service.

5.3 Software Stack

Table 5.1 Software stack and versions

Component	Version	Role in the system
Python	3.10 (3.10.12 used for training)	Scoring service, model training, statement parsing
FastAPI	≥ 0.115 , < 1.0	HTTP scoring service on port 8000
Uvicorn	≥ 0.30	ASGI server
Pydantic	≥ 2.7	Request validation, including the amount and text bounds of Section 7.6
scikit-learn	≥ 1.5 (1.7.2 used for training)	HistGradientBoostingClassifier, isotonic calibration, IsolationForest

Component	Version	Role in the system
NumPy / pandas	>= 1.26 / >= 2.2	Feature construction, dataset generation, statement tables
joblib	>= 1.4	Model serialisation
SHAP	>= 0.45	Per-prediction attribution for the explain endpoint
NetworkX	>= 3.2	Bipartite graph views and ring detection on contributed edges
pdfplumber / pypdf	>= 0.11 / >= 4.2	Statement extraction, primary and fallback
bcrypt / PyJWT	>= 4.1 / >= 2.8	Password hashing and bearer tokens
SQLite	Bundled with Python	Single store for profiles, reputation, edges, graph cache, users
Node.js	20 LTS or newer	Client build and the Express layer
React / TypeScript	18.3 / 5.4	Single-page client
Vite	5.2	Build tool and development server
Tailwind CSS	4.0	Styling
Recharts	2.12	Charts in the dashboard views
jsQR	1.4	QR decoding from an uploaded image or the device camera
Express	4.19	Authentication pass-through, upload proxy, static hosting
Capacitor	8.5	Android packaging
pytest	>= 8.0	581 automated tests
Streamlit	>= 1.35	Optional analyst dashboard (dashboard/)

5.4 Functional Requirements

16. Register and authenticate an account identified by its UPI ID, with no claim of address verification unless a provider confirms it.
17. Import a bank or UPI statement (PDF or CSV) and build a behavioural profile from it, keeping spend and credit rows distinct.
18. Accept a payee as a pasted UPI ID, a phone number, a scanned QR image or a camera capture, and extract the address from it.
19. Return a pre-payment assessment containing a risk score, a confidence score, an evidence level, a verdict, one typed row per evidence family and the list of families that could not be read.
20. Record a confirmed payment so that the payee reputation and graph structures grow, and allow a payee to be reported.
21. Answer a user's questions about a verdict through an assistant that can explain but cannot decide.

5.5 Non-Functional Requirements

- Latency: a complete pre-payment assessment within a few hundred milliseconds on the development environment (measured in Section 8.6).

- Safety: no write of any kind on the assessment path; the store is modified only when a payment is confirmed or a report is filed.
- Honesty: no response field may present an unmeasured quantity as a measurement, and no verdict vocabulary may exist outside the single mapping.
- Security: bearer-token authentication on every user-scoped endpoint, ownership checks on record-level reads, bounded request text, and no internal exception text in a client response.
- Portability: the trained artefact must load on a machine whose NumPy major version differs from the training machine's.

6. METHODOLOGY

The method is organised around a single question asked before a payment: on the evidence that can actually be gathered right now, how bad does this look, and how much evidence is there? Sections 6.1 to 6.3 establish what the system may claim about an entity. Sections 6.4 to 6.6 define evidence, its availability and its combination. Sections 6.7 to 6.10 describe the individual intelligence layers.

6.1 Entity Status Model

A payee address occupies exactly one of four states, and the distinction between them is the foundation of every claim the system makes:

Status	Meaning	What may be claimed
UNKNOWN	No user of this deployment has ever paid or reported this address.	Nothing. The absence is reported as absence.
OBSERVED	At least one confirmed payment or imported statement row names this address.	Counts and shape, to the extent the sample supports them.
REGISTERED	An account exists in this system for this address.	That an account exists - not that it is safe or that the address is genuine.
VERIFIED	A payment service provider confirmed the address exists.	Existence, as confirmed by the named provider, with the time of confirmation.

Two rules follow and are enforced in code. REGISTERED is not safety: anyone can register, and a scammer registering an account must not thereby acquire a clean reading. UNKNOWN is not fraud: most first payments are to addresses the system has never seen, and treating novelty as guilt would make the system useless. A payee with fewer than five distinct payers is treated as having no readable shape, because a repeat ratio computed over two payers is arithmetic rather than measurement.

6.2 Evidence Levels

Coverage is reported as FULL, PARTIAL or MINIMAL, derived from which families could be read. The three core families - payee history, payer behaviour and the payer-payee relationship - are the ones whose absence drops an assessment out of FULL, because they are the families that distinguish a stranger from a known-good counterparty.

6.3 Evidence Families

Ten families are evaluated independently. Each returns a score in 0-100, an availability flag, a severity, a human-readable message and a set of typed facts. The confidence weights answer a single question - how much less would we know without this family - and they are weights on evidence value, not on risk.

Table 6.1 Evidence families, confidence weights and the facts they may declare

Family	Weight	What it reads	Facts it may declare
payee_history	24	Aggregate reputation: age, distinct payers, repeat ratio, reports filed	payee_shape, payee_reports
relationship	18	Whether this payer has paid this payee before	payee_familiarity
payer_behaviour	16	Amount, hour and velocity against this payer's own baseline	amount_deviation, hour_anomaly, velocity_daily, payer_reliability
ml_classifier	12	The calibrated behavioural classifier over all twelve features	attributed by counterfactual probing
address_and_qr	11	VPA structure, PSP handle validity, Unicode confusables, display-name mismatch, embedded URLs	address_integrity
graph	8	Position in the payer-payee graph: component size, bounded two-hop payer overlap	graph_position
amount_context	5	Size against the rail's cap and against what a stranger plausibly receives	amount_absolute
message_pressure	3	Coercive framing in the message that prompted the payment	message_coercion
link_safety	2	Reputation and structure of URLs carried in the payload or message	link_reputation
stated_intent	1	Consistency of the payer's stated purpose with who receives the money	stated_purpose

6.4 Availability Semantics: the Difference Between Zero and Unknown

This is the project's central technical commitment, and it is worth stating precisely because an earlier version of the pipeline violated it. Three payee features - age in days, distinct payers and repeat ratio - were substituted with the constants 180, 8 and 0.4 whenever the payee was unknown. Those three numbers describe a moderately established payee with a modest but real track record, so every address the system had never seen, which includes every mule account opened last week, was scored as though it had one.

The pipeline now preserves a three-way distinction:

- A measured value, including 0.0, which means “we looked and the answer is none”.
- NaN, which means “we could not look”. HistGradientBoostingClassifier reads NaN natively, routing it down whichever branch the training data supports, and the generator hides payee history on 24.5% of rows so that unavailability is a condition the model has actually learned a response to rather than a gap papered over.
- A companion feature, `payee_history_available`, which states explicitly which case holds, so that a repeat ratio of 0.0 and an absent repeat ratio are never the same input.

The guarantee is enforced rather than documented: the Evidence record refuses at construction time to hold an unavailable family that carries a score or declares a fact, so “we do not know”

cannot push a score in any direction anywhere downstream. On the wire, a non-finite internal value is published as null, never as a bare NaN token and never as zero.

6.5 Score Combination

Family scores are merged so that the strongest signal leads and the others add a damped amount:

```
ordered = sorted(available_scores, descending)
total   = ordered[0] + 0.4 * sum(ordered[1:])
risk    = min(99, round(total + agreement_bonus))
```

Straight addition saturates on any two moderate signals; a plain maximum throws corroboration away. The rule is monotonic by construction - every term is non-negative, so a further suspicious family can only raise the total or leave it unchanged - and an unavailable family is absent from the input rather than entering as a zero.

6.6 Fact-Aware Corroboration

Agreement between independent streams is worth a great deal, because independent streams rarely err together. Counting modules made that easy to fake: the classifier reads `is_night`, the payer rules evaluate `hour < 6` or `hour >= 22`, and those are the same predicate over the same timestamp. A payment could therefore collect a corroboration bonus for the system having looked at the clock twice.

Each family therefore declares the facts its score rests on, drawn from a closed vocabulary of fourteen observation types, and the bonus is indexed by the number of distinct facts bounded by the number of reporting families:

```
AGREEMENT_BONUS = {2: 6, 3: 14, 4: 22, 5: 28, 6: 32}
bonus = AGREEMENT_BONUS[min(len(distinct_facts), len(reporting_families))]
# participation requires an available family scoring >= 20
```

Both bounds are needed. Without the family bound, one family observing a payment's size and its hour would be paid as though two modules had agreed - which turned an ordinary monthly rent payment into a warning the first time corroboration was implemented. Without the fact bound, five families reading the same reputation row would be paid for five agreements about one thing.

6.7 Verdict Mapping

There is one mapping from a score to a band, in one module, and every band in the system comes from it:

```
score >= 70 -> BLOCK      # do not pay
score >= 45 -> STEP_UP    # verify the payee before paying
score >= 22 -> WARN       # check first
otherwise   -> APPROVE    # nothing suspicious found
```

Two policies sit beside it. A hard-block finding - a malformed address, a blocked payee - is a BLOCK whatever the arithmetic says, because that override is about findings rather than about scores. And the legacy LOW/MEDIUM/HIGH label that older screens still render is computed

from the band rather than from its own thresholds, so a payment cannot be published as BLOCK and MEDIUM in the same response, which is exactly what happened when the two vocabularies had separate ladders.

Figure 6.1 Evidence assembly and the verdict pipeline



6.8 Amount Context

Three parts of the system compare a payment against an amount, and they do so against deliberately different statistics. The classifier divides by the payer's median payment, because that is the statistic it was trained on. The payer-behaviour rules apply a fixed ladder - fifteen times, eight times, three times - to the payer's mean, because a fixed threshold ladder must be calibrated against the statistic it is applied to. The third part, amount_context, judges the amount absolutely: against the rail's cap for the payee type, and against what a stranger plausibly receives.

The coexistence is measured rather than assumed. On the demonstration payer, whose spending is bimodal - daily payments around Rs 220 and rent at Rs 18,500 - the mean is Rs 1,523 and the median Rs 222. The monthly rent is 12 times the mean and lands in the middle band; judged against the median it is 83 times, hits the top band, and turns a recurring payment into a monthly

warning. Worse, against the median both the rent at 83x and outright fraud at 416x sit in the same band, so the ladder loses resolution exactly where it needs it. The classifier does not have this problem because it learned the relationship instead of being handed a threshold.

Both comparisons are gated on the payer having made at least five recorded payments. Below that the baseline is itself the outlier: a payer whose only prior payment was Rs 10 has a median of Rs 10, so an ordinary Rs 5,000 transfer reads as 500 times their norm. The rules therefore report the comparison as unavailable rather than producing arithmetic, and the classifier family reports itself unavailable too - deliberately rather than by passing NaN, because the model reads that ratio as a real number and was not fitted with it missing.

6.9 Incremental Graph Intelligence

Fraud is relational: a collection account is characterised by many payers who each pay once, by a short life, and by sharing its payer pool with other young addresses. Reading that from a rebuilt graph per request does not fit a real-time budget, so the structures are maintained instead of computed:

- A persisted union-find table over payer and payee nodes, so that a component size is a lookup rather than a traversal. Observations only ever add edges, which is precisely the condition under which union-find is sufficient.
- A per-payee cache row holding payer count, repeat-payer count, arrival span, activity span and payment count, refreshed when a payment is observed.
- A bounded two-hop query that finds the peer payee sharing the most payers with this one, with both sides required to have at least ten payers before overlap is scored.

That last gate exists because of two false positives found by an audit of this layer. A new shop sharing four customers with a popular delivery service was blocked at a risk of 70, and a household of five people paying two new local shops was blocked at 91. Both are ordinary neighbourhood behaviour; the shape that actually distinguishes a ring needs a larger shared pool on both sides. The graph family also scores position only - the payee's shape is reported but carries no weight there, because the payee-history family already scores shape, and having both score it inflated a legitimate new shop from a step-up to a block.

The read path is write-free by construction. An earlier version created its tables on read, which is both a write on a read path and, on a network-mounted database file, enough to produce I/O errors outright; the schema now belongs to the connection helper, and a missing table is reported as the feature being unavailable.

6.10 Payload, Link and Message Analysis

The address itself carries evidence. The address family checks the structure of the virtual payment address, validates the PSP handle against the known set, folds Unicode confusables to catch look-alike handles - a Cyrillic character that renders identically to its Latin counterpart survives

Unicode normalisation and must be detected by confusable folding, not by NFKC - and compares the display name in a QR payload against the address it accompanies. URLs carried in the payload or in the message are scored separately. Coercive framing is scored by a small lexical model over the supplied message, and the stated purpose is checked against who actually receives the money: a payment described as an electricity bill going to a personal phone-number address is a mismatch worth reporting.

6.11 Machine Learning Pipeline

6.11.1 Dataset

No public labelled UPI fraud corpus exists, so the training data is simulated - the approach PaySim established. The property that matters is that labels come from the generative process and not from a rule over the features: each fraud scenario is a distinct way of behaving, and the model must recognise it from the same features the live system can compute. The generator builds 2,000 synthetic payers with lognormal spending (median about Rs 270) of whom 12% are marked as making large payments routinely - rent, fees, deposits - and then draws 120,000 payments at a 1.2% fraud rate across four scenarios: social engineering 50%, mule collection 22%, account takeover 20% and card testing 8%.

Overlap between the classes is deliberate. Legitimate payments reach the Rs 1,00,000 cap, because real people pay rent and tuition over UPI; a quarter of account takeovers go to a payee the victim has paid before; a third of social-engineering payees are aged accounts, bought or stolen precisely so the payee looks established. Two artefacts that had made the problem separable were found and removed during development: fraud rows carried a fractional distinct-payer count where legitimate rows carried integers, so a single split on “is this a whole number” separated the classes perfectly; and amounts topped out near Rs 61,000, so no legitimate payment reached the top of the legal range and “amount is large” became a near-perfect fraud rule.

6.11.2 Features and Availability Mask

Table 6.2 Feature set and availability

Feature	Can be NaN	Definition at decision time
amount	No	The payment amount in rupees
amount_over_user_avg	No (family reports unavailable)	Amount divided by the payer's median payment
is_night	No	1 if the hour is before 06:00 or at or after 22:00
hours_from_usual	No	Circular distance from the payer's most active hour
seconds_since_last_txn	No	Gap since this payer's previous payment
txns_last_hour	No	This payer's payments in the preceding hour
txns_today_over_avg	No	Payments today divided by the payer's daily average
payee_is_new	No	1 if this payer has not paid this payee before

Feature	Can be NaN	Definition at decision time
payee_age_days	Yes	Days since the payee was first observed
payee_distinct_payers	Yes	Distinct payers observed paying this payee
payee_repeat_ratio	Yes	Repeat payers over distinct payers (needs ≥ 5 payers)
payee_history_available	No	1 if the payee reputation row could be read, else 0

The mask that hides payee history is a saturating function of payer count - a payee with nine distinct payers is observed about half the time, and coverage rises from there - rather than a cutoff, because a hard threshold puts a discontinuity in the data that a tree will happily learn as a fact about the world. Two properties are required and both are tested: the mask must be realistic, and it must not encode the label. The residual correlation is published as an availability bias of 0.72 percentage points (Section 8.5), which is small enough that “unknown” cannot serve as a proxy for “fraud” and honest about the fact that an address too new to have been observed genuinely is somewhat more likely to be a collection account.

6.11.3 *Model, Calibration and Threshold*

- Estimator: HistGradientBoostingClassifier, 300 iterations, learning rate 0.08, 31 leaves, balanced class weights, wrapped in isotonic calibration with five-fold cross-validation, because boosted trees are systematically over-confident and the score is published as a probability.
- Baseline: a logistic regression pipeline with median imputation and standardisation, which cannot represent absence at all and so must be told the payee is average - the failure this project exists to remove, kept as a measured comparison rather than an assertion.
- Unsupervised second opinion: an IsolationForest fitted on legitimate traffic only, because an anomaly detector should model normal rather than a mixture of normal and fraud.
- Splits: 60% train, 15% validation, 30% test by row count (72,000 / 18,000 / 30,000), stratified throughout, because at a 1% positive rate an unstratified fold can contain almost no fraud.
- Operating point: the threshold that maximises recall within a 1% false-positive budget is chosen on the validation split and then measured once on test. Choosing it on test - as an earlier version did - reports the best of many thresholds as though it were unbiased.

One portability measure is worth recording. A fitted HistGradientBoostingClassifier carries a fit-only random generator that pickles as a PCG64 bit generator, and NumPy changed how those pickle between major versions, so the artefact failed to load on a machine whose NumPy differed from the training machine's. That state is stripped before serialisation, and the pipeline asserts the artefact loads under a different NumPy major version.

6.11.4 *The Classifier as One Family*

The classifier contributes score but is deliberately capped below the block threshold when it is the only family with anything to say: it can reach a step-up on its own, but blocking a payment on a

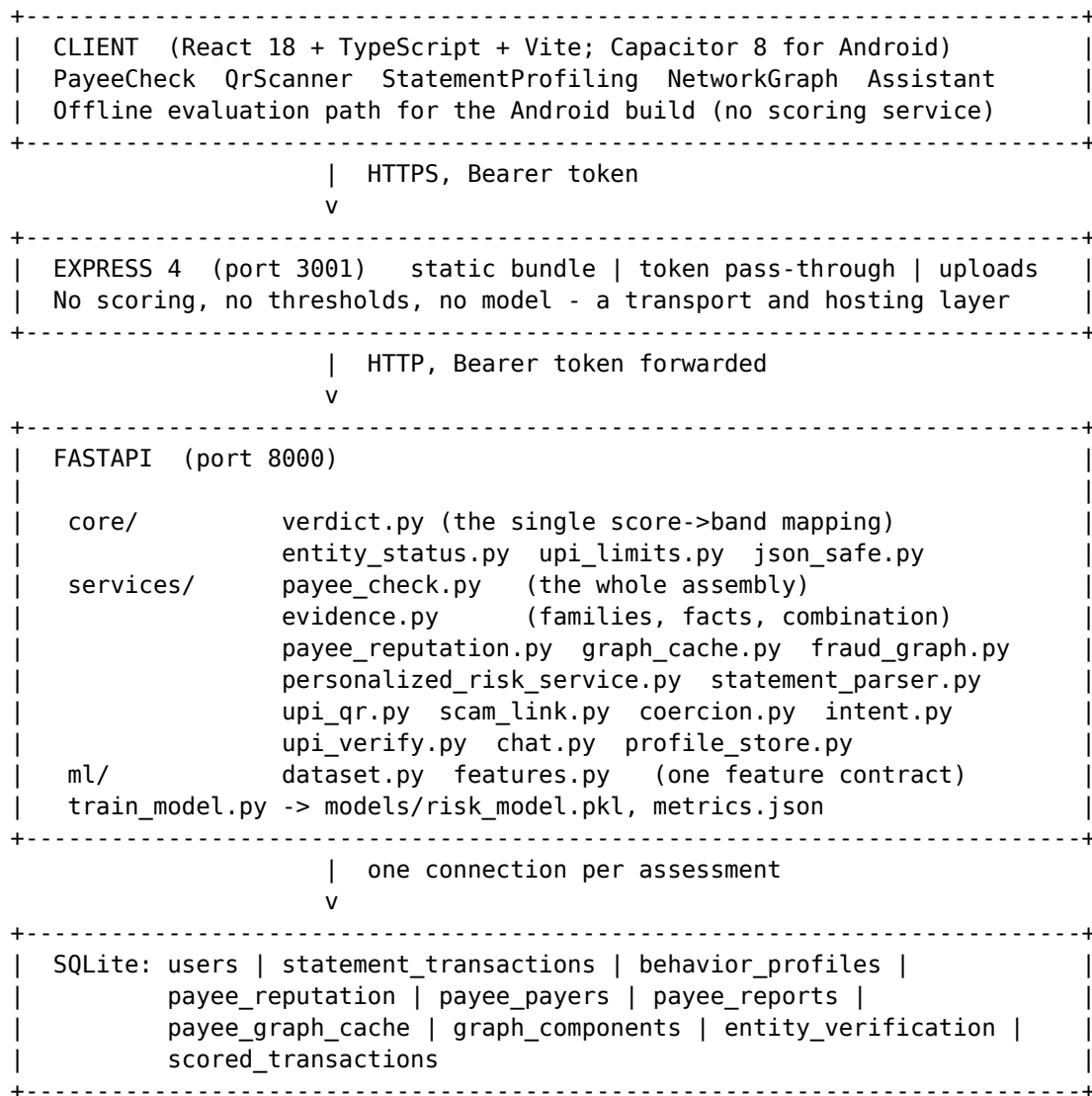
single model reading, with no deterministic family able to name a reason, is not a decision a user can act on. Its facts are attributed by counterfactual probing - each candidate fact is neutralised in a copy of the feature vector and the prediction re-run - so that the facts it declares for corroboration are the ones its reading actually rests on, rather than a static list that would let one family collect the maximum bonus alone.

7. ARCHITECTURE (IMPLEMENTATION AND TESTING)

7.1 System Architecture

Three processes and one store. The client is a React and TypeScript single-page application. The Express layer serves that bundle, holds no analysis logic of its own, and forwards API calls to the scoring service with the caller's bearer token attached. The FastAPI service performs every assessment and owns the model. A single SQLite file holds users, imported statement rows, behavioural profiles, payee reputation, contributed edges, the graph structures and filed reports.

Figure 7.1 System architecture



7.2 Data Model

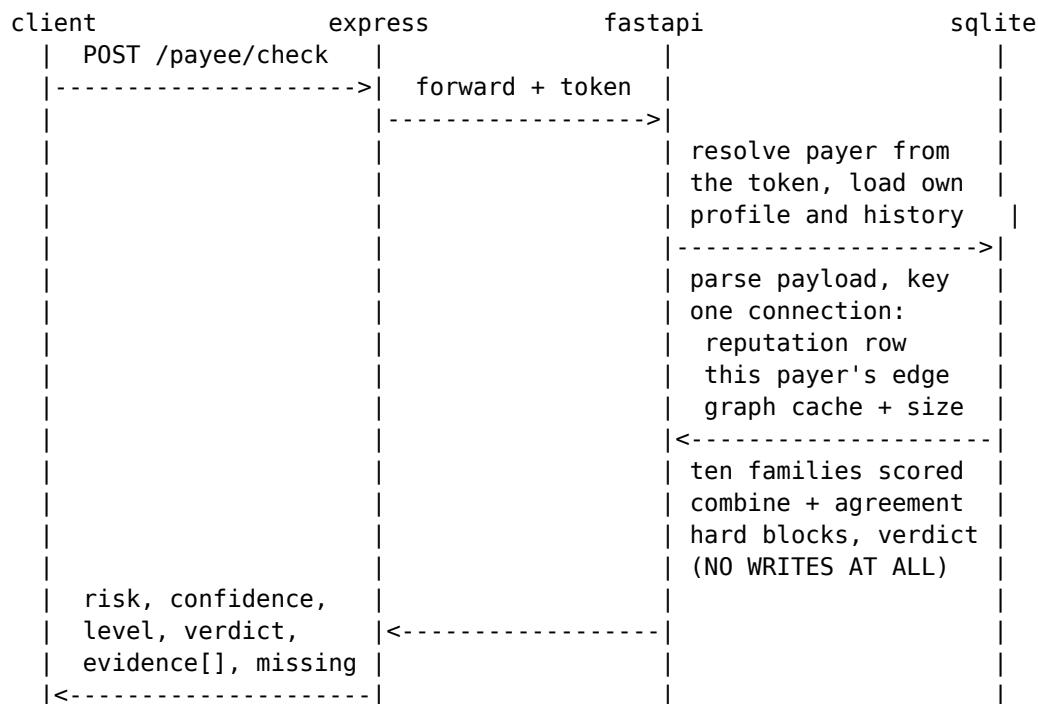
Table 7.1 Database schema

Table	Purpose and notable columns
users	Account per UPI ID; bcrypt password hash; provider verification state
statement_transactions	Imported rows: timestamp, amount, merchant, upi_id, status, txn_type (DEBIT /

Table	Purpose and notable columns
	CREDIT kept distinct)
behavior_profiles	Derived per-payer profile: counts, mean, median, max, most active hour, night rows, known payees and handles
payee_reputation	Per-address aggregates: first_seen, last_seen, payment_count, total and squared amounts, min, max
payee_payers	One row per (payee, payer): payment count, first and last time - the contributed edge table
payee_reports	Reports filed against an address, idempotent per reporter
payee_graph_cache	Maintained graph features per payee: payer count, repeat payers, arrival span, activity span, payments
graph_components	Persisted union-find: node, parent, component size
entity_verification	Provider confirmations: who verified, when, by what method
scored_transactions	Scored payments from the monitoring endpoint; primary key is a full UUID

7.3 The Pre-Payment Check Path

Figure 7.2 Pre-payment check sequence



Identity is resolved by the service from the bearer token, never taken from the request body: the profile and history handed to the assessment are exactly what that token is entitled to read, so a payer identifier in a body cannot widen what gets read. One database connection serves the whole assessment; three were opened before, which on a network-mounted file was both slower and the difference between working and not.

7.4 API Surface

Endpoint	Auth	Purpose
POST /auth/register, /auth/login	None	Account by UPI ID; bcrypt; bearer token issued
POST /payee/check	Required	The pre-payment assessment. No writes.
POST /payee/confirm	Required	Record that the payer proceeded; grows reputation and graph
POST /payee/report	Required	File a report against an address; idempotent per reporter
POST /statement/upload	Required	Import a statement (10 MB, 20,000 row caps) and rebuild the profile
POST /personalized-risk-check	Required	The payer-behaviour family alone, for the older screen
POST /predict	Required	Score and store a transaction; returns the feature vector
GET /transactions, /heatmap	Required	The caller's own scored payments
GET /explain/{tx_id}, /behavior/{tx_id}	Required	SHAP attribution and behavioural detail, owner-only
GET /fraud-graph, /fraud-rings, /gnn-fraud-detection	Required	Contributed-graph views and ring detection
GET /model-drift, /temporal-patterns	Required	Drift report and the caller's temporal pattern
POST /chat, GET /chat/status	Required	Explain-only assistant; facts assembled server-side
GET /health, /payee/intents	None	Liveness, model loadability, the intent vocabulary

The response contract is fixed and carries six fields: `risk_score`, `confidence_score`, `evidence_level`, `verdict`, `evidence` (one typed row per family, available or not) and `missing_evidence`. The property worth stating plainly is that a family which could not look is visible in the response and invisible in the arithmetic.

7.5 Client Implementation

- **PayeeCheck**: the main screen. Paste an address, scan a QR from the camera or an uploaded image, optionally give an amount, a purpose and the message that prompted the payment; renders the verdict, the per-family evidence rows including the unavailable ones, and what the corroborating observations were.
- **QrScanner**: jsQR over a camera stream or an uploaded image; the extracted address is then checked through the same endpoint as a pasted one.
- **StatementProfiling**: statement upload, extraction warnings, and the derived profile.
- **NetworkGraph** and **FraudRings**: the contributed payer-payee graph and detected rings, with payer identities shown as opaque account identifiers.
- **Assistant**: questions about the current verdict, answered by the explain-only service.
- **Android**: the same bundle in a Capacitor 8 shell, with an offline evaluation path that mirrors the server's band thresholds from a single shared module so the two cannot disagree.

7.6 Security Boundary

- Authentication: bcrypt password hashing, HS256 bearer tokens with a 24-hour lifetime, and a secret required from the environment at import time - the process refuses to start without it.
- Authorisation: every user-scoped endpoint requires a token, verified by an unauthenticated sweep in the test suite; record-level endpoints check ownership and answer 404 rather than 403, so an identifier's existence is not confirmed to a stranger.
- Input bounds: the payload is capped at 4,096 characters, an address at 255, a message at 2,000, a report reason at 500, an assistant question at 4,000 and its history at 20 turns. A one-megabyte payload was previously accepted, echoed back in full, and would have become the primary key of a reputation row.
- Control characters, NUL in particular, are refused in addresses and messages: a NUL is where a C string ends, so two different payees can become one in a log, a filename or the next service along.
- Validation failures return 422 with the location and message but without echoing the offending value, so declining a megabyte does not mean returning it.
- Uploaded filenames are reduced to a name on both POSIX and Windows spellings before a retained copy is written; the multipart header previously reached the filesystem as given, which is a path and not a name.
- Errors: a corrupt upload is a 400 with an actionable message, and the library's own exception text is logged rather than published.
- Secrets: no credential is hardcoded; configuration is read from a gitignored .env; the assistant's key is masked wherever it is printed.
- SQL: every statement is parameterised, and the assessment path performs no writes.

7.7 Testing Strategy

The suite contains 581 tests across 33 files, written to pin safety properties rather than happy paths. Several exist because a property was violated during development and the test is what makes that violation impossible to repeat.

Table 7.2 Test suite composition

Test module	Tests	What it holds in place
test_evidence_model, test_evidence_availability	33	Unavailable families carry no score; NaN and measured zero stay distinct
test_api_contract	27	The six canonical fields; every family present exactly once; the fields the client reads exist
test_graph_cache, test_graph_semantics	39	Incremental structures; read path issues no writes; the two false positives stay fixed
test_fact_corroboration	19	Distinct facts, family-bounded bonus, no double counting

Test module	Tests	What it holds in place
test_ml_family	18	The classifier as a family: solo ceiling, unavailability, no self-corroboration
test_amount_context	39	Cold start, outliers, train/serve parity, current payment excluded from its own baseline
test_verdict_vocabulary	27	Six boundary cases; one mapping; the legacy label cannot disagree
test_api_boundary_security	28	Input bounds, control characters, filename reduction, error shape, auth sweep
test_predict_serialization	16	Non-finite values become null; internal NaN preserved; no global encoder change
test_entity_status	14	Four states; REGISTERED is not safety; UNKNOWN is not fraud
test_model_pipeline, test_model_portability	~20	No feature separates the classes; artefact loads across NumPy majors
Parsing, QR, links, coercion, geo, device, chat, others	~300	Statement extraction, payload analysis and the supporting services

Three disciplines are worth naming because they changed the results reported in Chapter 8. First, tests point at the store: a session fixture redirects the database path to a temporary file, because the suite was silently exercising the real one. Second, when a correctness fix lowered a published metric, the lower number was adopted - the availability work cost 4.2 points of PR-AUC and 6.5 points of recall, and those are the figures reported. Third, every audit of this system was required to produce a measurement rather than an argument; three of them found that the expected fix would have made things worse, and were abandoned on the evidence.

8. RESULT AND ANALYSIS

All figures below come from models/metrics.json, written by the training run, and from a scripted end-to-end demonstration. The data is simulated: no public labelled UPI fraud corpus exists, and these numbers describe the method on that data rather than field performance. The dataset holds 120,000 rows at a 1.183% fraud rate, split 72,000 / 18,000 / 30,000, with 355 fraudulent payments in the test split.

8.1 Classifier Performance

Table 8.1 Classifier performance on the held-out test split

Metric	Logistic baseline	Selected model	Reading
ROC-AUC	0.9410	0.9652	Optimistic at a 1% base rate; reported for comparability
PR-AUC	0.3037	0.4860	The informative view under this imbalance
Recall at 1% FPR budget	27.89%	49.30%	Of the fraud present, what is caught inside the budget
Realised FPR	0.97%	0.95%	Measured on test at the validation-chosen threshold
Precision at that point	25.52%	38.38%	Roughly two alerts per true positive
Operating threshold	0.0987	0.1174	Chosen on validation, measured once on test
Brier score	0.00942	0.00798	Against 0.01169 for always predicting the base rate
Brier skill score	0.1944	0.3174	Calibration gain over that reference

The selected model is the calibrated histogram gradient-boosted ensemble. It improves PR-AUC by 60% over the linear baseline and nearly doubles recall inside the same false-positive budget. The gap is instructive rather than incidental: the baseline must impute the payee features it cannot read, and being told that an unknown payee is average is exactly the failure this project set out to remove. The unsupervised IsolationForest, fitted on legitimate traffic only, reaches a ROC-AUC of 0.8544 - useful as a second opinion on normality, and clearly weaker than the supervised model, which is the expected ordering.

8.2 Recall by Fraud Scenario

An averaged recall figure hides the case this project exists for, so recall is reported per scenario.

Table 8.2 Recall by fraud scenario at the operating threshold

Scenario	n	Baseline	Selected model	Character of the case
account_takeover	73	78.08%	94.52%	Burst to fresh payees, often at night, amount pushed up
card_testing	20	100%	100%	Tiny probe payments

Scenario	n	Baseline	Selected model	Character of the case
				in quick succession
mule_collection	80	13.75%	63.75%	Young payee, many one-time payers
social_engineering	182	6.04%	19.23%	The payer authorises it; nothing about the payer is wrong

The ordering is the honest result. Where the payer's own behaviour is disturbed - a takeover, a probe burst - the model is strong. Where the payer behaves normally by construction, it catches roughly one in five. That is not a defect to be tuned away; it is the measurement that justifies the other nine evidence families, and it is why a single-model architecture would be the wrong answer to this problem. Mule collection improves fourfold over the baseline precisely because the payee features are read as measurements when available and as absence when not.

8.3 Where the Payment Is Stopped: Evidence Families Versus the Model

Because the classifier is one family among ten, a payment the model cannot see can still be stopped, and a payment the model flags can still be cleared by corroborating evidence being absent. The demonstration in Section 8.6 shows both directions happening on the same statement.

8.4 Feature-Half Ablation

The project's claim is that scoring the payer is not enough. The ablation measures it by fitting the same estimator on each half of the feature set.

Table 8.3 Feature-half ablation

Feature set	PR-AUC	Recall at 1% FPR	Social-engineering recall
Payer features only (7)	0.3175	29.01%	3.85%
Payee features only (5)	0.1495	24.23%	11.54%
Both halves (12)	0.4860	49.30%	19.23%

Using both halves raises social-engineering recall 7.69 percentage points above the better half alone, and PR-AUC by 0.17 over the better half. One caveat is recorded in the metrics file and repeated here rather than hidden: in this simulator a social-engineering payment is often well above the victim's usual amount, because real scam amounts sometimes are, so the payer half is not entirely blind to the case. The stronger claim - that payer features cannot see authorised fraud at all - is not supported by this data, and is not made. What the numbers support is the combined lift.

8.5 Evidence Availability

Quantity	Value	Why it is reported
Rows with payee history hidden	29,376 (24.48%)	The condition the live system is usually in
Fraud rate when hidden	1.729%	An unobserved payee is somewhat more likely to be fraudulent

Quantity	Value	Why it is reported
Fraud rate when available	1.005%	The comparison
Availability bias	0.724 pp	Must stay small, or 'unknown' becomes a proxy for 'fraud'

The bias is real, deliberate and bounded. It is real because an address too new to have been observed genuinely is more often a collection account; deliberate because removing it would require a mask that contradicts how coverage actually works; and bounded by a test, because a large value would mean the model had learned to warn on every genuine first payment to a stranger - which is most first payments. The correct response to an unreadable payee is to fall back on the payer's own behaviour, not to read a number the system made up.

8.6 End-to-End Demonstration

A scripted demonstration builds a 288-row synthetic PhonePe-style statement for one payer - median payment Rs 222, most active hour 19:00 - seeds 30 payees with 15,973 crowd payments, and then scores nine payments through the complete assembly, not through the model alone. Four are ordinary payments; five are planted fraud patterns. The verifier derives the velocity features from the statement itself rather than carrying hand-written values, so the only way to change them is to change the statement.

Table 8.4 End-to-end demonstration results (complete assembly, not the model alone)

Case	Payment	Amount (Rs)	Model p	Risk	Conf.	Level	Verdict
--	Canteen lunch	90	0.001	0	96	FULL	APPROVE
--	Weekly groceries	640	0.004	12	96	FULL	APPROVE
--	Monthly rent	18,500	0.000	20	96	FULL	APPROVE
--	Cab to the office	120	0.000	0	96	FULL	APPROVE
F1	Probe burst, 3rd of 8	5	0.024	14	96	FULL	APPROVE (see 8.7)
F2	Funnel account, 3rd of 5	4,500	0.325	99	96	FULL	BLOCK (model-led)
F3	Hijacked profile, 2nd of 3	92,400	0.007	74	96	FULL	BLOCK (other families)
F4	Victim-authorized transfer	18,000	0.007	58	96	FULL	STEP_UP (other families)
F5	Look-alike bank handle	47,500	0.220	99	96	FULL	BLOCK (model-led)

Three results in this table matter more than the verdicts themselves. F3 and F4 are stopped while the model scores them at 0.007 - below its alert threshold - because the payee is days old, has taken money from a handful of payers and returned nothing to any of them; that is the payee check doing the work the classifier cannot. F2 and F5 are the mirror image: the deterministic families see a plausible payment and the model's reading is what carries the verdict past the block threshold. And the four ordinary payments - including the Rs 18,500 monthly rent, which is 83 times this payer's median - are approved, which is the property a fraud system is usually worst at.

8.7 A Negative Result, Reported

F1, a Rs 5 probe payment, is not flagged. The model scores it 0.024 against a 0.1174 threshold and the assembly returns APPROVE at a risk of 14. The cause was established by measurement rather than assumed. The demonstration statement plants eight probe payments three minutes apart, and the case is scored at the third of them, where only two payments precede it inside the hour. The training scenario for card testing sits at a median gap of about 35 seconds with roughly seven payments in the preceding hour; fed those inputs, the same artefact scores the same payment at 0.45 to 0.53 and flags it comfortably.

So the system detects card testing - per-scenario recall on that class is 100% in Table 8.2 - and what this row shows is that the third payment of a 21-minute run is not yet the shape the scenario describes. It is recorded here as an open item rather than corrected by rewriting the demonstration to suit the model, and Section 9.2 lists the two honest ways to close it.

8.8 Latency

A complete assessment - address parsing, ten families, one database connection, the calibrated model, combination and verdict - was measured over 25 consecutive runs in the development environment with the store on a network-mounted filesystem: median 110 ms, 95th percentile 144 ms, minimum 83 ms, maximum 198 ms. On a local disk the database portion is faster. This is within the budget a payment screen can absorb before the user presses pay, which was the design constraint on the graph layer: the maintained structures answer in a lookup what a rebuild could not answer at all.

8.9 Defects Found by Audit, and What They Cost

Six audits were run against the frozen architecture. They are reported here because a project report that lists only successes describes a system nobody tested.

Audit	Defect found	Resolution
Availability	Unknown payees imputed as moderately established	NaN end to end; PR-AUC fell 0.528 to 0.486 and the lower figure was adopted
Feature overlap	Correlated families counted as independent agreement	Typed facts, family-bounded bonus; no verdict changed on the measured cases
Graph semantics	Two false positives: a new shop, and a household of five	Peer gate and a minimum shared pool of ten payers on both sides
Amount context	A payer with one prior payment scored 500x their norm	A five-payment floor; risk 56 STEP_UP fell to 22 WARN, confidence 88 to 76
Verdict vocabulary	A monitor endpoint and the mobile path had their own bands	One mapping; the offline mirror pinned to it by test
API boundary	Unbounded text, NUL accepted, client filename used as a path, NaN in JSON	Bounds, control-character refusal, filename reduction, null for non-finite values

8.10 Limitations

- The evaluation data is simulated. Every metric in this chapter describes the method on that data, and the simulator's realism is an assumption, not a result.
- The system holds no NPCI, PSP or banking integration. Payee knowledge is only what this deployment's own users have imported or confirmed, so a genuinely new address is unknown to it - and is reported as unknown.
- Social-engineering recall is 19.23%. The payee-side evidence raises what the assembly catches, as F3 and F4 show, but one in five is the measured model figure and it is not presented as more.
- Provider verification is optional and unconfigured by default; without it the system reports an address as unverified rather than claiming it checked.
- The contributed graph is visible in aggregate to any authenticated user. Payer identities are opaque account identifiers, but the set of addresses one anonymous payer paid is visible, which is a linkability exposure recorded in the security audit and not yet closed.
- The coercion and intent models are small and lexical; they detect framing patterns, not meaning, and a carefully worded scam message will pass them.

9. CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This project set out to move fraud detection on UPI from after the payment to before it, and to do so without pretending to information the system does not have. Both halves of that were necessary. The first is a consequence of the rail: a payment that cannot be reversed can only be prevented, and prevention has to happen in the seconds while the payer is still looking at the screen. The second turned out to be the harder engineering problem and the more interesting one.

The system that resulted assesses a payee address before any money moves, using ten independent evidence families whose scores are merged by a damped-maximum rule and whose agreement is rewarded only when it rests on genuinely distinct observations. It publishes two numbers rather than one - how bad the available evidence looks, and how much evidence there was - and one verdict in one vocabulary, produced in one place. A family that cannot measure something reports itself unavailable, and the record that carries a family's contribution refuses at construction time to hold an unavailable family with a score, so "we could not look" cannot become "nothing is wrong" anywhere downstream.

The measured results support a specific and bounded claim. The calibrated classifier reaches a PR-AUC of 0.486 and catches 49.3% of simulated fraud within a 0.95% false-positive rate, with 94.5% recall on account takeover and 19.2% on victim-authorised social engineering; using both halves of the feature set raises the latter 7.7 points above the better half alone. In the end-to-end demonstration, two planted frauds are stopped by the model when the deterministic families see nothing, and two others are stopped by the payee evidence while the model scores them below its threshold - which is the architectural argument of the project, visible in four rows of one table. Ordinary payments, including a monthly rent 83 times the payer's median, are approved.

Equally, what the project does not show is stated plainly. The data is simulated. There is no rail-level or bank-level integration. One in five is the model's reach on the hardest and most common fraud class in India, and a Rs 5 probe payment in the demonstration is not flagged, for a reason that was measured and is reported rather than smoothed over. The value of the work is not a headline accuracy figure; it is an architecture in which every number can be traced to something that was actually observed, and in which the absence of a number is itself reported.

Along the way the project also produced a small amount of evidence about how to build systems of this kind. Six audits were run against a frozen architecture, and three of them found that the obvious fix would have made the system worse - switching the amount ladder to a median denominator, changing the voting design, and tightening the graph's overlap rule all failed on measurement. The discipline of requiring an audit to produce a number rather than an argument is, in retrospect, the most transferable result here.

9.2 Future Scope

9.2.1 *Immediate*

22. Close the F1 demonstration gap honestly, by either scoring the probe burst at its sixth to eighth payment, where the run genuinely reaches the velocity the scenario describes, or tightening the planted spacing so the third payment matches it. Retraining is not required: train and serve feature definitions were verified identical, and the same artefact flags the case when fed the velocity the scenario implies.
23. Correct the same-day velocity count in the demonstration harness, which counts payments later in the day as well as earlier ones, and the equivalent in the monitoring helper for client-supplied timestamps.
24. Close the graph linkability exposure: pseudonymise per response, or restrict the aggregate graph view to the caller's own neighbourhood.

9.2.2 *Data and Model*

25. Replace the simulator with real labelled data. The feature contract is the interface: any source producing the twelve columns plus a label trains the same pipeline unchanged. A bank or PSP partnership, or an anonymised complaints corpus with confirmed outcomes, would convert every figure in Chapter 8 from a statement about a method into a statement about the world.
26. Model the payee's inflow as a sequence rather than as aggregates. Arrival bursts, the rate at which a new address accumulates one-time payers and the shape of its first week are temporal patterns that the current five aggregate features flatten.
27. Evaluate a graph neural network on the contributed graph once it is large enough for the comparison to mean anything, against the maintained-feature baseline rather than against nothing.
28. Add drift monitoring with action: the report exists, but nothing currently retrains or alerts on a sustained population shift.

9.2.3 *System*

29. Confirmation of payee through a provider integration, which would make VERIFIED reachable in normal operation rather than only when a verification endpoint is configured.
30. Federated or privacy-preserving reputation sharing between deployments, so that a young collection account seen by one community warns another without either sharing payer identities.
31. On-device scoring for the Android build, so that the assessment survives a dead network without falling back to a thinner heuristic path.
32. A reporting loop that feeds confirmed outcomes back as labels, which is the only route to supervised learning on real behaviour that a payer-side system can own.

9.2.4 Evaluation

33. A user study on the verdict language. STEP_UP is the band that asks the payer to do something, and whether it produces verification or annoyance is an empirical question this project has not answered.
34. Adversarial evaluation: a scammer who knows the rules will age an account, recruit repeat payers and avoid confusable handles, and the system's resistance to that has not been measured.

10. REFERENCES

- [1] National Payments Corporation of India, “UPI product statistics,” NPCI, Mumbai, 2026. [Online]. Available: <https://www.npci.org.in/what-we-do/upi/product-statistics>
- [2] Business Standard, “UPI sets new record at 24.51 bn transactions in August; value at Rs 29.82 trn,” 1 September 2026. [Online]. Available: https://www.business-standard.com/finance/news/upi-transactions-august-2026-record-volume-npci-126090100506_1.html
- [3] National Payments Corporation of India, “Revision in UPI transaction limits for person-to-merchant categories, effective 15 September 2025,” NPCI circular, 2025 (as reported in Outlook Money and Paytm Blog, September 2025).
- [4] Ministry of Finance, Government of India, “Reply to Lok Sabha Unstarred Question No. 2432 on digital payment fraud data and RBI anti-fraud measures,” 3 August 2026.
- [5] LocalCircles, “UPI fraud survey: one in five households reported experiencing UPI fraud in the last three years,” 2025. [Online]. Available: <https://www.localcircles.com>
- [6] Y. Z. Toh, “Combating authorized push payment scams in fast payment systems,” Payments System Research Briefing, Federal Reserve Bank of Kansas City, 2024.
- [7] “‘Authorized push payment’ bank fraud: what does an effective regulatory response look like?,” Journal of Financial Regulation, vol. 10, no. 2, 2024.
- [8] E. A. Lopez-Rojas, A. Elmir and S. Axelsson, “PaySim: a financial mobile money simulator for fraud detection,” in Proc. 28th European Modeling and Simulation Symposium (EMSS), Larnaca, Cyprus, 2016, pp. 249-255.
- [9] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson and C. E. Leiserson, “Anti-money laundering in bitcoin: experimenting with graph convolutional networks for financial forensics,” in Proc. KDD Workshop on Anomaly Detection in Finance, Anchorage, AK, USA, 2019.
- [10] B. Hooi, H. A. Song, A. Beutel, N. Shah, K. Shin and C. Faloutsos, “FRAUDAR: bounding graph fraud in the face of camouflage,” in Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 2016, pp. 895-904.
- [11] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng and P. S. Yu, “Enhancing graph neural network-based fraud detectors against camouflaged fraudsters,” in Proc. 29th ACM Int. Conf. on Information and Knowledge Management (CIKM), 2020, pp. 315-324.
- [12] F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation forest,” in Proc. 8th IEEE Int. Conf. on Data Mining (ICDM), Pisa, Italy, 2008, pp. 413-422.
- [13] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T.-Y. Liu, “LightGBM: a highly efficient gradient boosting decision tree,” in Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 3146-3154.
- [14] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in Proc. 22nd Int. Conf. on Machine Learning (ICML), Bonn, Germany, 2005, pp. 625-632.
- [15] J. Davis and M. Goadrich, “The relationship between precision-recall and ROC curves,” in Proc. 23rd Int. Conf. on Machine Learning (ICML), Pittsburgh, PA, USA, 2006, pp. 233-240.

- [16] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” PLOS ONE, vol. 10, no. 3, e0118432, 2015.
- [17] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 4765-4774.
- [18] V. Chandola, A. Banerjee and V. Kumar, “Anomaly detection: a survey,” ACM Computing Surveys, vol. 41, no. 3, pp. 1-58, 2009.
- [19] Unicode Consortium, “Unicode security mechanisms,” Unicode Technical Standard #39. [Online]. Available: <https://www.unicode.org/reports/tr39/>
- [20] “Detection of mule accounts and fraudsters in UPI transactions using AI and machine learning techniques,” IRE Journals, 2024. [Online]. Available: <https://www.irejournals.com/paper-details/1716166>
- [21] “Enhancing UPI fraud detection: a machine learning approach using stacked generalization,” International Journal of Mathematics, Statistics and Management (IJMSM), vol. 2, no. 1, 2025.
- [22] “UPI fraud detection using machine learning,” International Journal of Innovative Research in Technology (IJIRT), paper 178524, 2025; and International Journal of Research and Analytical Reviews (IJRAR), paper 25B3295, 2025.
- [23] “A comprehensive survey on machine learning techniques for UPI and telecommunication fraud detection,” International Journal of Engineering Research and Technology (IJERT), 2024.
- [24] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., “Scikit-learn: machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
- [25] A. A. Hagberg, D. A. Schult and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX,” in Proc. 7th Python in Science Conference (SciPy), Pasadena, CA, USA, 2008, pp. 11-15.
- [26] Reserve Bank of India, “Annual report and reports on trend and progress of banking in India: chapters on payment systems and frauds,” RBI, Mumbai, 2025-26.

Project artefacts. The implementation accompanying this report comprises approximately 22,600 lines of Python, TypeScript and TSX across the scoring service, the client and the tooling, with 581 automated tests. The trained model is regenerated with a single command (python backend/train_model.py), which writes models/risk_model.pkl, models/isolation_forest.pkl, models/shap_background.json and models/metrics.json - the file from which every figure in Chapter 8 is taken.